4.5 Logging in Spring Boot Applications

• What is Logging in Spring Boot?

- Logging means recording important events happening inside your application.
- Instead of printing: `System.out.println("User created");`
- We use: `log.info("User created");`

• Why Logging is Important:

- In real systems we can't debug using console, We need the history of what happened.
- Logging helps in:
  - Debugging issues.
  - Tracking API calls.
  - Monitoring system behavior.
  - Finding production bugs.
  - Auditing actions.
- **Example:**

```
log.info("Creating student with name: {}", name);
log.error("Error while creating student", e);
```

- Output**:**

```
2026-02-05 10:30 INFO  Creating student with name: Siba
2026-02-05 10:30 ERROR Error while creating student
```

• What are logging frameworks in Java ?

- Java provides different **logging frameworks** to help developers log application events.
- Instead of building logging from scratch, we use **ready-made libraries** like:
  - JUL (java.util.logging)
  - Log4j
  - Logback
  - TinyLog
- **java.util.logging (JUL) (Built-in) :**
  - It comes with Java (no dependency needed).
  - But it is hard to configure and less powerful.
- **Log4j (Old Popular Framework) :**
  - It is very powerful and flexible configuration.
  - It has some security issues (Log4Shell).
  - It is very old version, currently deprecated by java and replaced by **Log4j2**.
- **Logback (Modern & Fast) :**
  - Used by Spring Boot by default.
  - It's performance is high and also easy to configure.
  - It works with SLF4J
- **TinyLog (Lightweight) :**
  - It has very small library.
  - It used in small apps.

- Java provides frameworks like **JUL**, **Log4j**, and **Logback** to handle logging. In Spring Boot, we use **SLF4J** as an *abstraction* and **Logback** as the default *implementation*.

4.5 Logging in Spring Boot Applications

• SLF4J :

- SLF4J stands for **Simple Logging Facade for Java**.
- SLF4J is not a logging framework, It is a **logging abstraction** / **facade** (interface) for Java logging.
- It provides a common API for logging, while the actual logging is done by frameworks like:
  - Logback
  - Log4j2
  - JUL
- This allows switching between Logback, Log4j, or JUL without changing application code.

• Elements of a Logging Framework :

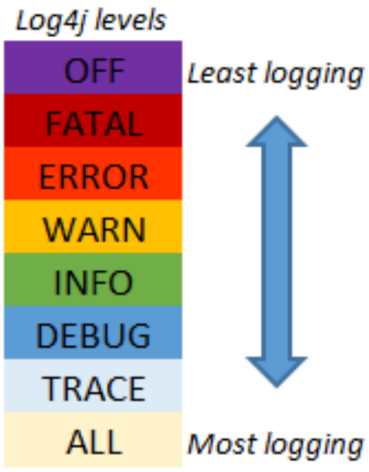
- Every logging framework consists of **three** core components that work together to **capture**, **process**, and **deliver** log messages efficiently.

1. **Logger :**
   - The **Logger** captures log events and messages generated by the application.
   - It acts as the entry point for the logging system.
2. **Formatter :**
   - The **Formatter** organizes and formats the log messages captured by the logger into a readable and standardized format.
3. **Handler(appender) :**
   - The **Handler** (also called an **Appender** in some frameworks) dispatches the formatted log messages to their destination, such as: *Console output*, *Log files*, *Databases*, *Monitoring systems*, *Email services*.

• Log Levels :

- Log messages can be categorized based on their **severity** and **purpose**.
- Spring Boot supports multiple logging levels that help developers monitor application behavior and troubleshoot issues effectively.
- **Logging Levels :**

1. **FATAL** – **Critical errors** that cause the application or system to terminate.
2. **ERROR** – **Runtime errors** or failures that prevent a specific operation from completing.
3. **WARN** – Indicates **potential issues** or unexpected situations that may require attention.
4. **INFO** – **Records** important application events and normal runtime operations.
5. **DEBUG** – Provides **detailed information** useful for debugging and understanding application flow.
6. **TRACE** – Captures very **fine-grained** and **detailed diagnostic** information for deep debugging.



- Logging frameworks follow a **hierarchical** log level system.
- When a log level is configured, all messages at **that level** and **higher severity** are logged, while **lower-severity** messages are **ignored**.
- This helps to optimize logging output and system performance.

4.5 Logging in Spring Boot Applications

• What is a Log Formatter ? :

- A **log formatter** defines the **structure** and appearance of log messages by adding metadata such as **timestamps**, **log levels**, **thread names**, and **logger names**.
- In Spring Boot, formatting can be configured using logging patterns or Logback configuration.

Ex.

```
log.info("Student created");
```

Formatter converts it into:

```
2026-02-05 12:30:10 INFO  StudentService : Student created
```

• How to Configure Log Format ? :

- Using **application.yml**

We can add something like this:

```
logging.pattern.console=%d{yyyy-MM-dd hh:mm:ss}  %5level  [%-15.15t] --- %-50.50c{1} : %m%n
```

- **Meaning :**

Pattern	Meaning
%d	Date/time
%t	Thread name
%level	Log level
%c	Class/logger name
%m	Actual message
%n	New line

- **Output:**

```
2026-05-26 09:38:34  ERROR  [main      ] --- c.s.p.p.c.StudentClientImpl            : Exception in StudentClientImpl.createStudent()
```

• What is a log handler / appender ? :

- A **log handler** (or appender) is the component responsible for **sending** formatted **log events** to destinations such as **console**, **files**, **databases**, or **remote logging systems**.
- Handlers also manage **buffering**, **asynchronous delivery**, **file rotation**, and **log routing**, making them a critical part of production logging systems.

• How set destination of logs :

- Add this in application.properties

```
logging.file.name=application.log
```

- Then it will create a file named as **application.log** inside **'target'** folder and start storing the logs inside that file.

4.5 Logging in Spring Boot Applications

• What are Rolling Logs ? :

- Rolling Logs is a mechanism used to automatically **rotate** log files based on:
  - File size
  - Time duration
  - Or both
- Instead of storing all logs in a single continuously growing file, the logging framework creates new log files **periodically** and **archives old ones**.

• Why Rolling Logs are Needed ? :

- In production systems, applications **continuously** generate **logs** for API requests, Authentication events, Database errors, Payment transactions, System warnings, Performance metrics and many more.
- If logs are stored in a single file, then the file size **grows** drastically (e.g. 10GB → 100GB → 1TB).
- This creates serious production problems:
  - Disk space exhaustion
  - Slow log searching
  - Server instability
  - Application crashes
  - Difficult maintenance
  - Large backup sizes
- **Rolling logs** solve these problems automatically.

• Types of Rolling Strategies :

1. **Time-Based Rolling :**
   - Creates new log files **after** a fixed **time interval**.
   - Typically used for **Daily** logs or **Hourly** logs.
   - Example:
     - aap-2026-05-25.log
     - app-2026-05-26.log
2. **Size-Based Rolling :**
   - Creates a new log file when current file **exceeds** a **configured size**.
   - Typically used when applications generate heavy traffic.
   - Example:
     - app.0.log
     - app.1.log
     - app.2.log
3. **Size + Time Based Rolling :**
   - Most production systems combine both **Daily rollover** and **Maximum file size limit**
   - This is considered the most stable production strategy.
   - Example:
     - app-2026-05-01.0.log
     - app-2026-05-01.1.log
     - app-2026-05-01.2.log
     - app-2026-05-02.0.log
     - app-2026-05-02.1.log

4.5 Logging in Spring Boot Applications

• Components of Rolling Logs :

- Rolling logs has 3 components, Appender, TriggeringPolicy, RollingPolicy.

1. **Appender :**
   - It is responsible for **writing log events** to a **specific destination** such as the console, file system, database, or external logging platform.
   - It acts as the **output layer** of the logging framework and **receives log events** generated by the **logger**.
   - In rolling logs, **RollingFileAppender** is commonly used to **write logs** into files while coordinating with **rolling policies**.

2. **TriggeringPolicy :**
   - It determines **when** a log rollover should occur.
   - It continuously evaluates conditions such as **file size limits**, **date changes**, or **time intervals** during **log write operations**.
   - Once the configured condition is satisfied, it **signals** the **logging framework** that **rollover** is **required**.

3. **RollingPolicy :**
   - It defines **how** the rollover process should be executed after a trigger occurs.
   - It is responsible for **renaming old** log files, **creating new** active log files, **archiving** logs, **deleting expired** logs, and **enforcing retention rules** such as **maxHistory** and **totalSizeCap**.
   - It essentially manages the complete **lifecycle** of **archived logs**.

• Logback Configuration for rolling logs :

```
<configuration>
  <appender name="ROLLING"
        class="ch.qos.logback.core.rolling.RollingFileAppender">
    <!-- Current active log file -->
    <file>logs/app.log</file>
    <!-- Rolling policy -->
    <rollingPolicy
      class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
      <!-- Archived log file pattern -->
      <fileNamePattern>
        logs/app-%d{yyyy-MM-dd}.%i.log
      </fileNamePattern>
      <!-- Rotate after 100MB -->
      <maxFileSize>100MB</maxFileSize>
      <!-- Keep logs for 30 days -->
      <maxHistory>30</maxHistory>
      <!-- Maximum total archived log size -->
      <totalSizeCap>5GB</totalSizeCap>
    </rollingPolicy>
    <!-- Log format -->
    <encoder>
      <pattern>
        %d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n
      </pattern>
    </encoder>
  </appender>
  <root level="INFO">
    <appender-ref ref="ROLLING"/>
  </root>
</configuration>
```

<file>logs/app.log</file> : Current active log file is named as app.log inside logs folder.

<fileNamePattern>...</fileNamePattern> : Pattern for archived logs, it decides how to name the files and which basis files become separate.

<maxFileSize>100MB</maxFileSize> : Maximum size before rollover. It allows files to grow up till the given size.

<maxHistory>30</maxHistory> : Number of days/files to retain. In pure size best rolling it represents the number of files otherwise it represents the time period.

<totalSizeCap>5GB</totalSizeCap> : Total storage limit for archived logs. If size of archived logs exceeds 5 GB then logger deletes older logs.

<encoder>
<pattern>...</pattern>
</encoder> : Define log format. Encoder transforms internal logging events into a formatted textual or binary representation before writing them to console, file, or network.

4.5 Logging in Spring Boot Applications

• What is a LoggingEvent :

- A LoggingEvent is an **internal object** created by the logging framework that contains metadata such as *timestamp*, *log level*, *thread name*, *logger name*, *message*, and *exception details* before being encoded and written by appenders.
- That object looks something like this,

```
LoggingEvent {

   timestamp: 2026-05-26T21:30:10,
   level: INFO,
   loggerName: "com.cms.service.StudentService",
   threadName: "main",
   message: "User created",
   formattedMessage: "User created with id=15",
   throwable: null,
   marker: null,

   mdc: {
      requestId: "abc123",
      userId: "15"
   }
}
```

• What is a Async Logging :

- Async Logging is a logging mechanism where log events are first placed into an **in-memory queue** and later processed by a **dedicated background thread.**
- It **reduces** request **latency** and **improves** application **throughput**.

• **Synchronous Logging:**	• **Asynchronous Logging:**
- Simple - Reliable - Slower	- Faster - Scalable - Higher Throughput - Possible Log Loss

- Most **high-traffic production systems** use Async Logging because application performance is usually more valuable than guaranteeing every non-critical log entry is written immediately.

```
<configuration>
   <appender name="FILE"
         class="ch.qos.logback.core.rolling.RollingFileAppender">
      <file>logs/app.log</file>
      <encoder>
         <pattern>
            %d %-5level %logger{36} - %msg%n
         </pattern>
      </encoder>
   </appender>
   <appender name="ASYNC"
         class="ch.qos.logback.classic.AsyncAppender">
      <queueSize>5000</queueSize>
      <neverBlock>true</neverBlock>
      <appender-ref ref="FILE"/>
   </appender>
   <root level="INFO">
      <appender-ref ref="ASYNC"/>
   </root>
</configuration>
```

<queueSize>5000</queueSize> : Now maximum 500 events queue can hold.

<neverBlock>true</neverBlock> : When queue is full it drop logs instead of blocking request threads.

Work flow

API Request
↓
Business Thread
↓
Logger
↓
Async Queue
↓
Response Returned
————————————————
Background Logging Thread
↓
RollingFileAppender
↓
File / Elasticsearch / Cloud